48. Локальные переменные, область видимости и продолжительность жизни

Оглавление:

1. Область видимости и продолжительность жизни
2. Сокрытие имен
3. Область видимости переменных
4. Параметры функций
5. Заключение
6. Тест

Область видимости и продолжительность жизни

Прежде чем мы начнем, нам нужно сначала разобраться с двумя терминами: область видимости и продолжительность жизни. **Область видимости** определяет, где можно использовать переменную. **Продолжительность жизни** (или **время жизни**) определяет, где переменная создается и где уничтожается. Эти две концепции связаны между собой.

Переменные, определенные внутри блока, называются **локальными переменными**. Локальные переменные имеют **автоматическую продолжительность жизни**: они создаются (и инициализируются, если необходимо) в точке определения и уничтожаются при выходе из блока. Локальные переменные имеют **локальную область видимости** (или **«блочную»**, т.е. они входят в область видимости с точки объявления и выходят в самом конце блока, в котором определены.

Например, рассмотрим следующую программу:

```
#include <iostream>

int main()
{
    int x(5); // переменная x создается и инициализируется здесь
    double y(3.0); // переменная y создается и инициализируется здесь

    return 0;
}
// x и y выходят из области видимости и уничтожаются здесь
```

Поскольку переменные `x` и `y`, определены внутри блока, который является главной функцией, то они обе уничтожаются, когда `main()` завершает свое выполнение.

Переменные, определенные внутри вложенных блоков, уничтожаются, как только заканчивается вложенный блок:

```
#include <iostream>

int main()
{
    int x(5); // переменная x создается и инициализируется здесь

    // начало вложенного блока
    double y(3.0); // переменная y создается и инициализируется здесь
    } // y выходит из области видимости и уничтожается здесь

    // переменная x не может быть использована здесь, так как она уничтожена!

    return 0;
} // x выходит из области видимости и уничтожается здесь
```

Такие переменные можно использовать только внутри блоков, в которых они определены. Поскольку каждая функция имеет свой собственный блок, то переменные из одной функции никак не соприкасаются и не влияют на переменные из другой функции:

```
#include <iostream>

void nameFunction()
{
    int value(5); // value переменная здесь

    // value можно использовать здесь

} // value выходит из области видимости и уничтожается здесь

int main()
{
    // value может использовать внутри этой функции
    nameFunction();

    // value здесь также использовать

    return 0;
}
```

В разных функциях могут находиться переменные или **параметры** с одинаковыми именами. Это хорошо, потому что не нужно беспокоиться о возможности возникновения **конфликтов имен** между двумя независимыми функциями. В примере, приведенном ниже, в обеих функциях есть переменные `x` и `y`. Они даже не подозревают о существовании друг друга:

```
#include <iostream>

// переменные x и y используются только внутри функции add()
int add(int x, int y) // параметры x и y функции add() создаются здесь
{
    return x + y;
} // параметры x и y функции add() уничтожаются здесь

// переменные x и y функции mult() можно использовать только внутри функции mult()
int mult()
{
    int x = 5; // переменная x в функции mult() создается здесь
    int y = 6;
    int const prod = mult(x, y) + add(x, y); // переменные x и y функции mult() создаются и переменные x и y функции add()
    return 0;
} // x и y уничтожаются здесь
```

Вложенные блоки считаются частью внешнего блока, в котором они определены. Следовательно, переменные, определенные во внешнем блоке, могут быть видны и внутри вложенного блока:

```
#include <iostream>

int main()
{
    // начало внешнего блока

    int x(5);

    // начало вложенного блока
    int y(7);
    // переменная x видна здесь, так как она была определена в внешнем блоке
    std::cout << x << " + " << y << " = " << x + y;
    } // переменная y уничтожается здесь

    // переменная y здесь больше использовать, поскольку она уже уничтожена!

    return 0;
}
```

Сокрытие имен

Переменная внутри вложенного блока может иметь то же имя, что и переменная внутри внешнего блока. Когда подобное случается, то переменная во вложенном (внутреннем) блоке «скрывает» внешнюю переменную. Это называется **сокрытием имен**:

```
#include <iostream>

int main()
{
    // начало внешнего блока
    int const orange(5); // создание переменной orange

    int const fruit(7);
    if (orange == 5) // обращение к внешней переменной orange
    {
        // начало вложенного блока
        int const orange(8); // создание новой переменной orange

        // Теперь переменная orange скрывает переменную orange из внешнего блока
        int const fruit(9); // переменная fruit также скрывает переменную fruit из внешнего блока

        std::cout << "oranges = " << orange << " and " << fruit << '\n'; // обращение к переменным orange и fruit из вложенного блока
        } // переменные fruit и orange выходят из области видимости и уничтожаются здесь

        // Переменная fruit из внешнего блока теперь снова доступна
        std::cout << "oranges = " << orange << " and " << fruit << '\n'; // обращение к переменным orange и fruit из внешнего блока
    }
    return 0;
}
```

Результат выполнения программы:

```
10
5
```

Здесь мы сначала объявляем переменную `oranges` во внешнем блоке. Затем объявляем вторую переменную `oranges`, но уже во вложенном (внутреннем) блоке. Когда мы присваиваем `oranges` значение `10`, то оно относится к переменной во вложенном блоке. После выхода этого значения (и окончания внутреннего блока), внутренняя переменная `oranges` уничтожается, оставляя внешнюю `oranges` с исходным значением (`5`), которое затем выводится. Результат выполнения программы был бы тот же, даже если бы мы назвали вложенную переменную по другому (например, `inOranges`).

Обратите внимание, если бы мы не определили вложенную переменную `oranges`, то идентификатор `oranges` относился бы к внешней переменной и значение `10` было бы присвоено внешней переменной:

```
#include <iostream>

int main()
{
    // начало внешнего блока
    int const orange(5); // создание переменной orange

    if (orange == 5) // обращение к внешней переменной orange
    {
        // начало вложенного блока
        // нет переменной orange, поэтому мы используем переменную orange из внешнего блока

        orange = 10; // мы присваиваем в внешней переменной orange, хотя мы и находимся во вложенном блоке

        std::cout << orange << std::endl; // выводим значение внешней переменной orange
    } // вложенная переменная orange скрыта 10 даже после того, как мы работаем во вложенном блоке

    std::cout << orange << std::endl; // выводим значение внешней переменной orange

    return 0;
}
```

Результат выполнения программы:

```
10
10
```

В обоих примерах на внешнюю переменную `oranges` никак не влияет то, что происходит с вложенной переменной `oranges`. Единственное различие между двумя программами — это то, к чему применяется выражение `oranges = 10`.

Сокрытие имен — это то, чего, как правило, следует избегать, поскольку оно может быть довольно запутанным!

Правило: Избегайте использования вложенных переменных с именами, идентичными именам внешних переменных.

Область видимости переменных

Переменные должны определяться в максимально ограниченной области видимости. Например, если переменная используется только внутри вложенного блока, то она и должна быть определена в нем:

```
#include <iostream>

int main()
{
    // Не определяйте x здесь

    {
    // Переменная x используется только внутри этого блока, поэтому определим ее здесь
    int const(7);
    std::cout << x;
    }

    // x больше больше нигде не используется, поэтому мы уничтожаем ее здесь

    return 0;
}
```

Ограничивая область видимости, мы уменьшаем сложность программы, поскольку число активных переменных уменьшается. Таким образом, легче увидеть, где какие переменные используются. Переменная, определенная внутри блока, может использоваться только внутри этого же блока (или вложенных в него подблоков). Этим мы упрощаем понимание и логику программы.

Если во внешнем блоке нужна переменная, то её необходимо объявлять во внешнем блоке:

```
#include <iostream>

int main()
{
    // Определяем x здесь, потому что она нам будет нужна как во внутреннем, так и во внешнем блоке
    int const(5);

    {
        int x;
        std::cin >> x;
        // Переменная y используется только внутри этого блока, поэтому определим ее здесь
        int const(7);
        std::cout << x + y;
    } // y больше не используется здесь

    std::cout << y; // x в переменной y уже нужна нам здесь

    return 0;
}
```

Это один из тех редких случаев, когда нам может понадобиться объявить переменную до её первого использования.

Правило: Определяйте переменные в наиболее ограниченной области видимости.

Параметры функций

Хотя параметры функций не определяются внутри основного блока (тела) функции, в большинстве случаев они имеют локальную область видимости:

```
int mult(int x, int y)
{
    // Переменные x и y доступны здесь
    // Переменная prod не доступна
    int mux = (x - y) * x - y; // mux определяется здесь
    return mux;
} // x, y и mux уничтожаются здесь
```

Заключение

Переменные, определенные внутри блоков, называются локальными переменными. Они доступны только внутри блока, в котором определены (включая вложенные блоки) и уничтожаются при завершении этого же блока.

Определяйте переменные в наиболее ограниченной области видимости. Если переменная используется только внутри вложенного блока, то и определять её следует внутри этого же вложенного блока.

Тест

Задание №1

Напишите программу, которая просит пользователя ввести два целых числа: второе должно быть больше первого. Если пользователь введет второе число меньше первого, то используйте блок и временную переменную, чтобы поменять местами пользовательские числа. Затем выведите значения этих переменных. Добавьте в свой код **комментарии**, объясняющие, где и какая переменная уничтожается.

Результат выполнения программы должен быть примерно следующим:

```
Введите число: 4
Введите большее число: 2
Меняем значения местами
Меньшее число: 2
Большее число: 4
```

> **Подсказка:** Чтобы использовать кириллицу, добавьте следующую строчку кода в самое начало функции main():
>
> `setlocale(LC_ALL, "rus");`

Задание №2

В чём разница между областью видимости и продолжительностью жизни переменной? Какую область видимости и продолжительность жизни по умолчанию имеют локальные переменные (и что это значит)?

Ответ №1

```
#include <iostream>

int main()
{
    // Для вывода кириллицы:
    setlocale(LC_ALL, "rus");

    std::cout << "Введите число: ";
    int smaller;
    std::cin >> smaller;

    std::cout << "Введите большее число: ";
    int larger;
    std::cin >> larger;

    // Если пользователь ввел число не так, как нужно:
    if (smaller > larger)
    {
        // Объявляем вспомогательную переменную
        int temp(smaller);
        smaller = larger;
        larger = temp;
    } // temp уничтожается здесь

    std::cout << "Меньшее число: " << smaller << "\n";
    std::cout << "Большее число: " << larger << "\n";

    return 0;
} // smaller и larger уничтожаются здесь
```

Ответ №2

Область видимости определяет, где переменная доступна для использования. Продолжительность жизни переменной определяет, когда переменная создается и когда уничтожается.

Локальные переменные имеют локальную (блочную) область видимости, доступ к ним осуществляется только внутри блока, в котором они определены.

Локальные переменные имеют автоматическую продолжительность жизни, что означает, что они создаются в точке определения и уничтожаются в конце блока, в котором определены.